

Why Your FastAPI Is Not Fast

A Deep Dive into Hidden Bottlenecks

Satyam Soni — PyCon Hong Kong 2026

Software Engineering & Platforms track

A real incident

A FastAPI service, rewritten from a synchronous Flask predecessor for "speed," went into production for a payment-adjacent workload.

Under real concurrency, it was **slower** than the Flask version it replaced.

`async def` was everywhere. The benchmarks (single-request, no load) looked great. Production did not.

The uncomfortable truth

FastAPI is fast — but only if you use it correctly.

The framework gives you the tools. It does not protect you from yourself.

Today: **five anti-patterns** that silently destroy FastAPI performance — each with real code, real profiling output, and a measured before/after fix, from a reproducible "Orders API" you can run yourself tonight.

Quick framing: what async actually buys you

- One event loop, one thread (by default). `await` is a **cooperative** yield point — nothing else runs until you hit one.
- `async def` does **not** mean non-blocking. It means *capable of* yielding control — if the code inside never awaits anything, it behaves exactly like a synchronous function that hogs the loop.
- This talk stays in that territory: what happens *inside* your route handlers and dependencies. (Tracing/observability is the neighboring talk in this track — not duplicated here.)

Anti-pattern 1

Blocking the event loop

The setup

Orders API needs to call an external payment gateway before confirming an order — a classic slow I/O dependency.

```
@router.get("/bad")
async def blocking_call():
    resp = requests.get(f"{GATEWAY_URL}/delay/1", timeout=10)
    return {"gateway_status": resp.status_code}
```

`async def` — but `requests.get()` is synchronous. It blocks the **entire** event loop, not just this request, for its full duration.

LIVE DEMO — feel it

```
time (for i in 1 2 3; do curl -s -o /dev/null $URL/demo/ap1/bad & done; wait)
time (for i in 1 2 3; do curl -s -o /dev/null $URL/demo/ap1/good & done; wait)
```

3 concurrent requests, each hitting a gateway with a 1s delay.

Endpoint	Measured wall-clock
bad — requests.get() inside async def	3.046s (serialized)
good — httpx.AsyncClient	1.040s (concurrent)
bridge — run_in_executor	1.039s (concurrent)

Real asyncio debug-mode output

```
PYTHONASYNCIODEBUG=1 uv run python scripts/asyncio_debug_demo.py
```

```
WARNING:asyncio:Executing <Task ...> took 1.056 seconds
```

Logged **only** for the blocking path. asyncio's own debug mode is telling you exactly where the loop got stuck — for free, no extra instrumentation.

The fix

```
@router.get("/good")
async def async_call(request: Request):
    client: httpx.AsyncClient = request.app.state.http_client
    resp = await client.get(f"{GATEWAY_URL}/delay/1")
    return {"gateway_status": resp.status_code}
```

- Rewrite the call with an async-native client (`httpx` , `asyncpg` , etc.)
- Can't rewrite it yet? `loop.run_in_executor(None, sync_call)` is a valid migration bridge — same concurrency win, keep the old library for now.

Anti-pattern 2

Dependency injection lifecycle misuse

The setup

```
async def get_bad_db_session():  
    engine = build_engine(get_settings())          # new pool, new TCP handshake  
    async with build_sessionmaker(engine)() as session:  
        yield session  
    await engine.dispose()
```

`Depends()` is powerful — but nothing stops you from building a brand-new engine or HTTP client **inside** the dependency, on every single request.

Measured overhead — 50 sequential requests

```
bad  (new engine+client per request): mean=17.48ms  p50=16.82ms  p95=19.52ms
good (shared singletons)             : mean=1.73ms   p50=1.59ms   p95=2.56ms
```

```
mean overhead: 15.74ms per request (~10x)
```

That 16ms is almost entirely the fresh asyncpg connection handshake + auth
your "bad" path pays on **every** request.

The fix

```
@asynccontextmanager
async def lifespan(app: FastAPI):
    engine = build_engine(get_settings())
    app.state.sessionmaker = build_sessionmaker(engine)
    app.state.http_client = httpx.AsyncClient(timeout=10.0)
    yield
    await app.state.http_client.aclose()
    await engine.dispose()

async def get_session(request: Request):
    async with request.app.state.sessionmaker() as session:
        yield session
```

Build once in the lifespan. Hand it out via `Annotated[T, Depends(...)]`.

Anti-pattern 3

SQLAlchemy lazy loading in async

Failure mode 1: it crashes

```
orders = (await session.execute(select(Order).limit(3))).scalars().all()
item_counts = [len(order.items) for order in orders] # 💥
```

```
{
  "error_type": "MissingGreenlet",
  "error": "greenlet_spawn has not been called; can't call
           await_only() here. Was IO attempted in an
           unexpected place?"
}
```

Real captured response from `curl .../demo/ap3/bad-crash?limit=3` — not a constructed example.

Failure mode 2: it "works" — and costs you N+1

```
for order in orders:
    items = await order.awaitable_attrs.items    # one query PER order
```

Real `echo=True` SQL log, 5 orders:

	Queries issued
<code>bad-n1 (awaitable_attrs loop)</code>	6 (1 for orders + 5, one per order)
<code>good (selectinload)</code>	2 (1 for orders + 1 batched <code>IN (...)</code>)

Both return **identical results** — this is purely a query-count problem, exactly the kind of bug that passes code review.

The fix

```
result = await session.execute(  
    select(Order).options(selectinload(Order.items)).limit(limit)  
)
```

Eager-load in the query that fetches the parent. For extra safety, set

`lazy="raise"` on the relationship in your model — any *accidental* future lazy touch fails loudly in dev instead of degrading silently in prod.

Anti-pattern 4

Pydantic validation overhead

The setup

```
def bad_transform(order) -> dict:
    raw = {"id": str(order.id), "items": [...], ...} # manual dict
    validated = BadOrder.model_validate(raw)
    dumped = validated.model_dump() # round-trip #1
    revalidated = BadOrder.model_validate(dumped) # round-trip #2 (!)
    return revalidated.model_dump()
```

Deeply nested models, chained `field_validator`s, and a redundant `model_dump()` → `model_validate()` round-trip — common in middleware/serializer layers that grew organically.

Measured — real cProfile, 200 orders × 20 repeats

	Total time	Function calls
bad	0.056s	204,481
good	0.030s	116,463

~1.9x slower, ~1.8x more function calls — from validating the same data twice.

The fix

```
class GoodOrder(BaseModel):  
    model_config = ConfigDict(from_attributes=True)  
    id: uuid.UUID  
    items: list[GoodOrderItem]  
  
def good_transform(order) -> dict:  
    return GoodOrder.model_validate(order).model_dump(mode="json")
```

- `from_attributes=True` — validate straight from the ORM object, no manual dict.
- Data that never leaves the process? Skip Pydantic — a `TypedDict` or `dataclass` is enough.

Anti-pattern 5

Connection pool starvation

The setup

```
create_async_engine(url, pool_size=5, max_overflow=10, pool_timeout=2)
```

Tutorial defaults. Fine for a laptop demo. Dangerous under real concurrency — every connection is a real TCP+auth handshake to Postgres, and the pool is a hard ceiling.

Measured — identical locust load, pool config is the only variable

```
locust --headless -u 100 -r 100 -t 20s    (100 users, near-instant ramp)
```

	pool config	requests	failures	median latency	throughput
bad	5 / +10 / 2s timeout	905	19 (2.1%)	2200ms	47.8 req/s
good	20 / +40 / 10s timeout	3370	0	530ms	179.3 req/s

Real traceback from every one of those 19 failures:

```
sqlalchemy.exc.TimeoutError: QueuePool limit of size 5 overflow 10  
reached, connection timed out, timeout 2.00
```


The fix

```
create_async_engine(  
    url,  
    pool_size=20, max_overflow=40,  
    pool_timeout=10, pool_recycle=1800,  
)
```

Size the pool for your actual expected concurrency. Pick `pool_timeout` deliberately. Set `pool_recycle` so idle connections don't go stale.

Same code, same query, same load — **3.7x throughput, zero failures.**

The checklist

1. **Event loop hygiene** — no sync I/O inside `async def`
2. **Dependency lifecycle** — build once in the lifespan, not per request
3. **ORM loading strategy** — eager-load, never lazy-touch in a loop
4. **Pydantic usage** — `from_attributes=True`, skip it for internal-only data
5. **Connection pool sizing** — for real concurrency, not tutorial defaults

Every item ties to a real captured measurement in the repo — not a vibe.

Profiling toolkit reference

Tool	Use it for
<code>py-spy</code>	Zero-instrumentation sampling profiler, flame graphs
<code>asyncio</code> debug mode	Catching blocking calls inside <code>async def</code>
<code>cProfile</code> + <code>pstats</code>	Function-level CPU cost
SQLAlchemy <code>echo=True</code>	Every SQL statement + query counts
<code>locust</code>	Realistic concurrent load, pool/timeout behavior

github.com/satyamsoni2211/fastapi-performance-deep-dive

Every number in this talk — reproducible.

`docker compose up` and it's all running on your machine.

Questions?